

Setlist | Assignment SE | Johannes R.

A web archive for electronic music DJ sets, where every mix carries its full tracklist and every track links out to where it can be heard & purchased.

Description

A DJ records a set and uploads it as an MP3 or WAV file. Visitors can then see which track plays at which minute, who made it, at what tempo, in which alphanumeric key, and where to find or purchase it.

From any track a visitor reaches the artist behind it and the other sets it appears in. The same track shows up in many mixes, which is what makes the archive worth browsing. Listening and reading need no account. However, registered users can additionally comment on any set and upload sets of their own.

Because every track carries its BPM, its key in Camelot notation, its length and its genre, a tracklist here reads as a mixing document rather than a list of names: the tempo curve across a set becomes visible, and neighbouring keys show whether two tracks blend harmonically. The DJ supplies these values on upload, as they already hold them in their own library (eg. Rekordbox). A track that is already in the archive is reused, not duplicated, so every new set tightens the network between the ones already there.

Main functionalities

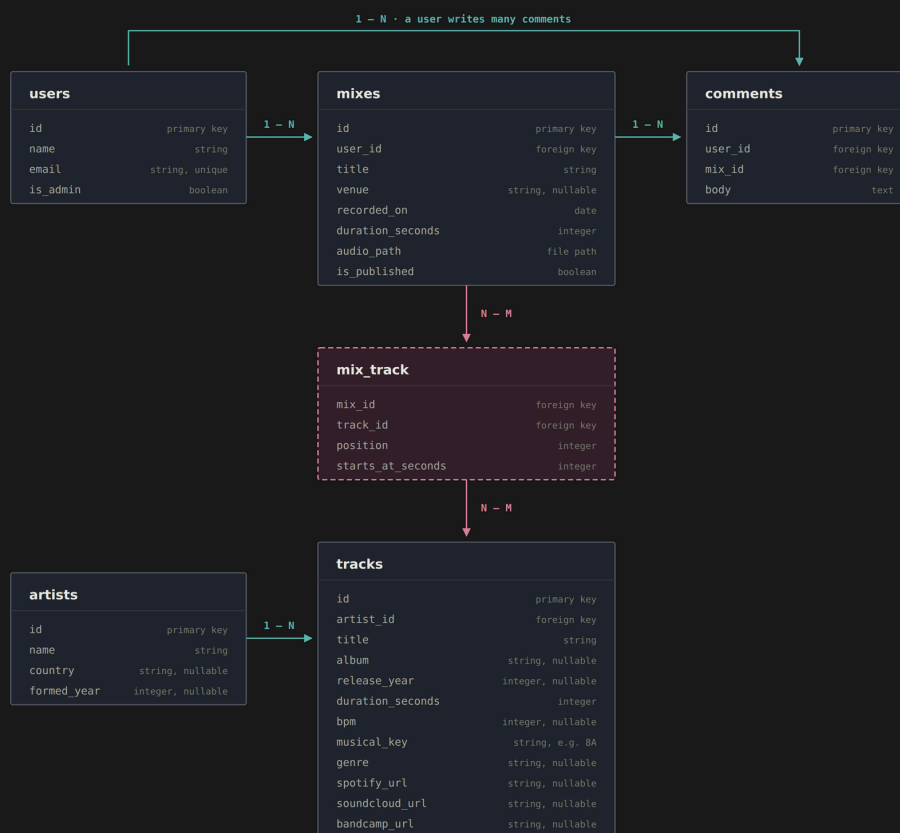
- Browse sets, tracks and artists through cross-linked detail pages
- Play a set while following its tracklist with start timestamps
- Read tempo, key, length and genre for every track in a set
- Follow any track to Spotify, SoundCloud or Bandcamp
- Registered users upload, edit and delete their own sets and build the tracklist
- Registered users comment on sets
- Admins manage the full catalogue and all user accounts

Role Allocation

ROLE	IN THE DATABASE	CAN DO
Visitor	Not stored [simply nobody logged in]	Browse sets, tracks and artists. Play any published set and read its tracklist. Writes nothing.
Registered user	A row in <code>users</code>	Everything a visitor can + comment on any set, and upload sets of their own.
DJ	Not a separate model. A registered user who has uploaded at least one mix	Upload, edit and delete their own sets and tracklists [only their own].
Admin	A row in <code>users</code> with <code>is_admin = true</code>	Manage every set, track, artist and account in the system.
Track-Artist	A row in <code>artists</code> — no account, never logs in	Nothing. Credited as the maker of a track; exists purely as reference data.

DJ is not a role in the database. Every registered user may upload, and the moment they do, the app credits them as the DJ of that mix. No extra column is needed. The question "is this user a DJ?" is answered by `$user->mixes()->exists()`. Their account name is the name shown on their sets.

Data Structure



→ one row on the left, many on the right

⋈ pivot table - carries the many-to-many relation

Models & Properties

Mix

One recorded DJ set, uploaded as an MP3 or WAV file.

id	PK
user_id	FK → users
title	string
venue	string, nullable
recorded_on	date
duration_seconds	integer
audio_path	string
is_published	boolean
timestamps	datetime

Track

One piece of music. Appears in any number of mixes.

id	primary key
artist_id	foreign key → artists
title	string
album	string, nullable
release_year	integer, nullable
duration_seconds	integer
bpm	integer, nullable
musical_key	string, Camelot
genre	string, nullable
spotify_url	string, nullable
soundcloud_url	string, nullable
bandcamp_url	string, nullable
timestamps	datetime

User

A registered account. Uploads mixes, writes comments.

id	primary key
name	string
email	string, unique
password	string, hashed
is_admin	boolean
timestamps	datetime

Artist

The musician behind a track. Reference data, no account.

id	primary key
artist_name	string
alias	string [optional]
country	string, nullable
formed_year	integer, nullable
timestamps	datetime

Comment

A reaction to a mix, written by a registered user.

id	primary key
user_id	foreign key → users
mix_id	foreign key → mixes
body	text
timestamps	datetime

mix_track

Not a model. It is the pivot table that carries the tracklist and is reached through the **Mix** ↔ **Track** relation, never as a class of its own. Its columns are shown in the data structure.

Model Relationships

User — Mix (1-N)

A `User` **hasMany** mixes

A `Mix` **belongsTo** a user (the DJ who uploaded it)

User — Comment (1-N)

A `User` **hasMany** comments

A `Comment` **belongsTo** a user

Mix — Comment (1-N)

A `Mix` **hasMany** comments

A `Comment` **belongsTo** a mix

Artist — Track (1-N)

An `Artist` **hasMany** tracks

A `Track` **belongsTo** an artist

Mix — Track (N-M)

A `Mix` **belongsToMany** tracks

A `Track` **belongsToMany** mixes

This relation is not stored on either table. It lives in its own pivot table, `mix_track`, which holds the two foreign keys plus `position` and `starts_at_seconds`, as the same track sits at a different place and starts at a different minute in every mix. Those two values belong to the combination of mix and track, not to either one alone. In Eloquent they have to be named explicitly with `withPivot()`, otherwise they are not loaded with the relation.

Pages (routes)

```
// public
welcome                -> homepage, landing page

mixes.index             -> list of all sets
mixes.show (id)         -> one set: player, tracklist, comments

tracks.index            -> list of all tracks
tracks.show (id)        -> one track: where to listen, in which sets it appears

artists.index           -> list of artists
artists.show (id)       -> one artist including their tracks

about · contact

// authentication
login · register · logout

// user area (logged in, own content only)
user.dashboard          -> overview of my uploaded sets

GET    user.mixes.index  -> list of my sets
GET    user.mixes.create -> empty form to upload a new set
POST   user.mixes.store  -> save the uploaded set
GET    user.mixes.edit (id) -> edit one of my sets, incl. its tracklist
PATCH user.mixes.update -> save the input from the edit form
DELETE user.mixes.destroy -> delete one of my sets

POST   mixes.comments.store -> post a comment on a set

// admin area (all content)
admin.dashboard

// Mix CRUD
GET    admin.mixes.index  -> list of all existing sets
GET    admin.mixes.create -> empty form to create a set
POST   admin.mixes.store  -> save the input from the create form
GET    admin.mixes.edit (id) -> edit an existing set
PATCH admin.mixes.update -> save the input from the edit form
DELETE admin.mixes.destroy -> delete an existing set

// Track CRUD (same routes as mixes)
admin.tracks.xxxxx

// Artist CRUD (same routes as mixes)
admin.artists.xxxxx

// User CRUD (same routes as mixes)
admin.users.xxxxx
```